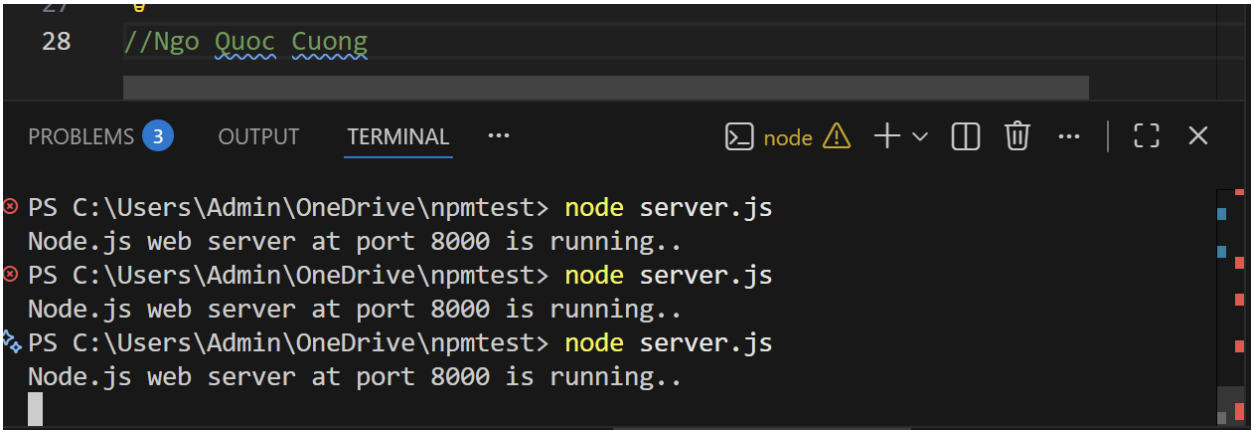
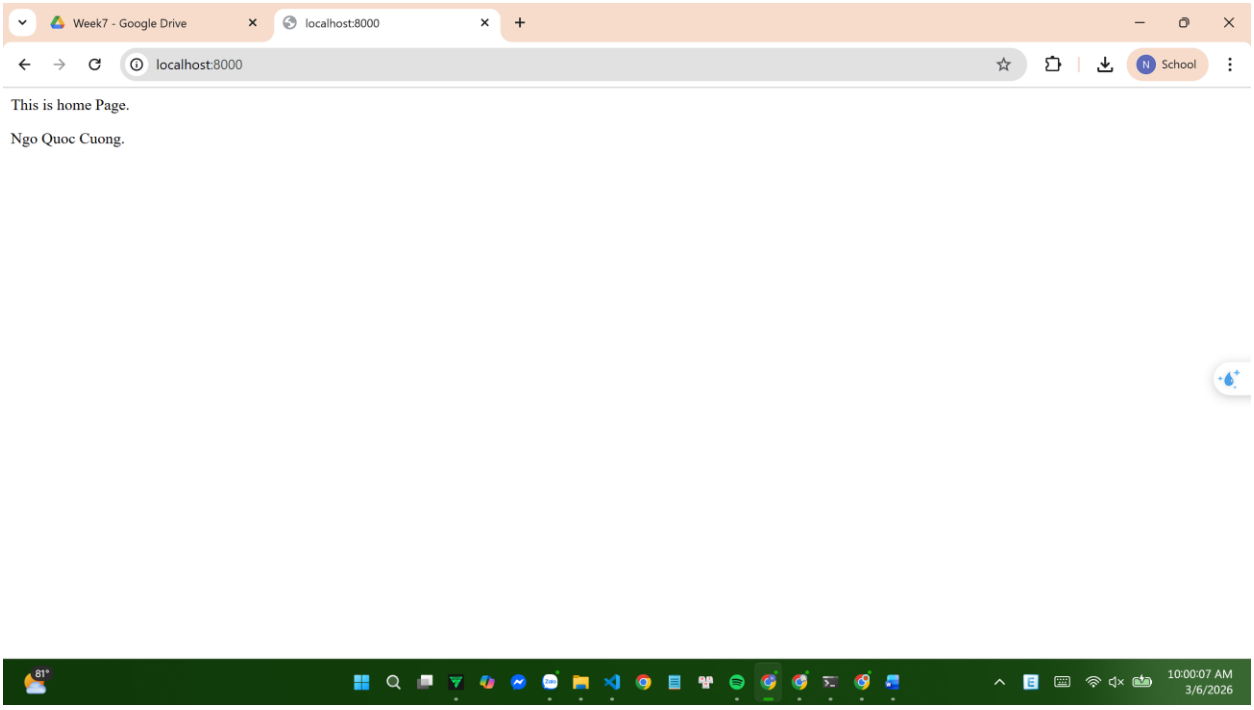
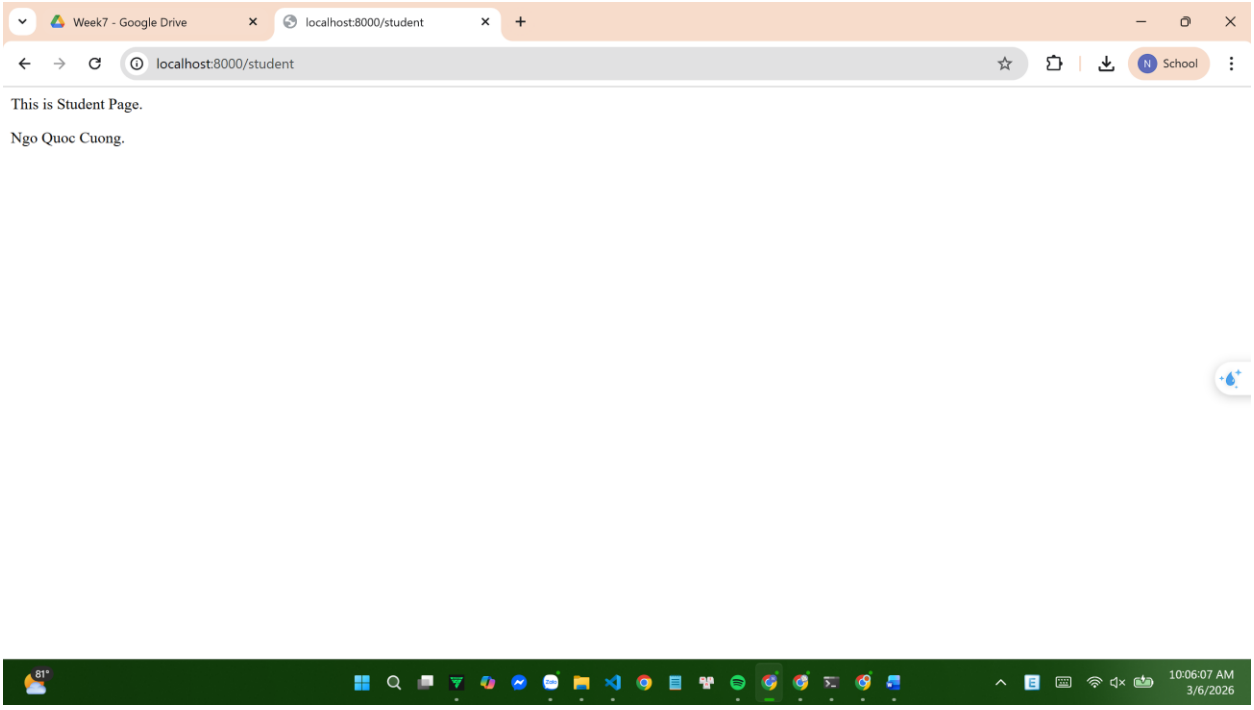
1. Screenshot 1:



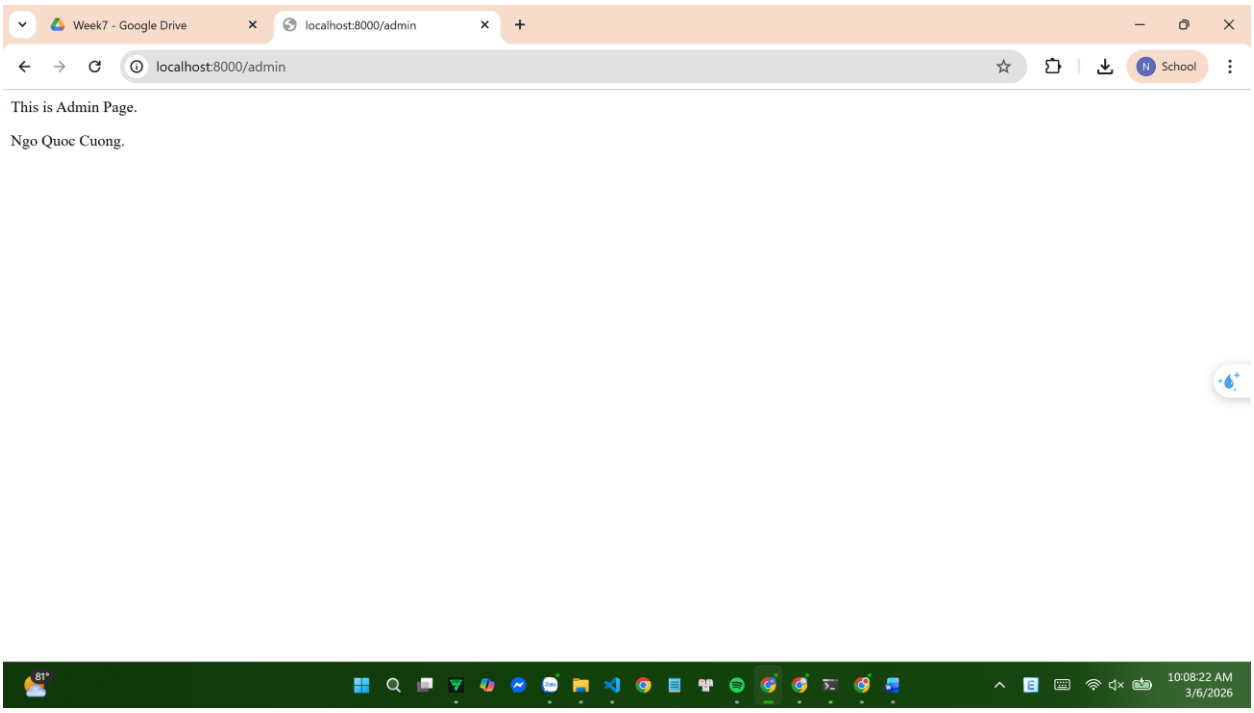
2. Screenshot 2:



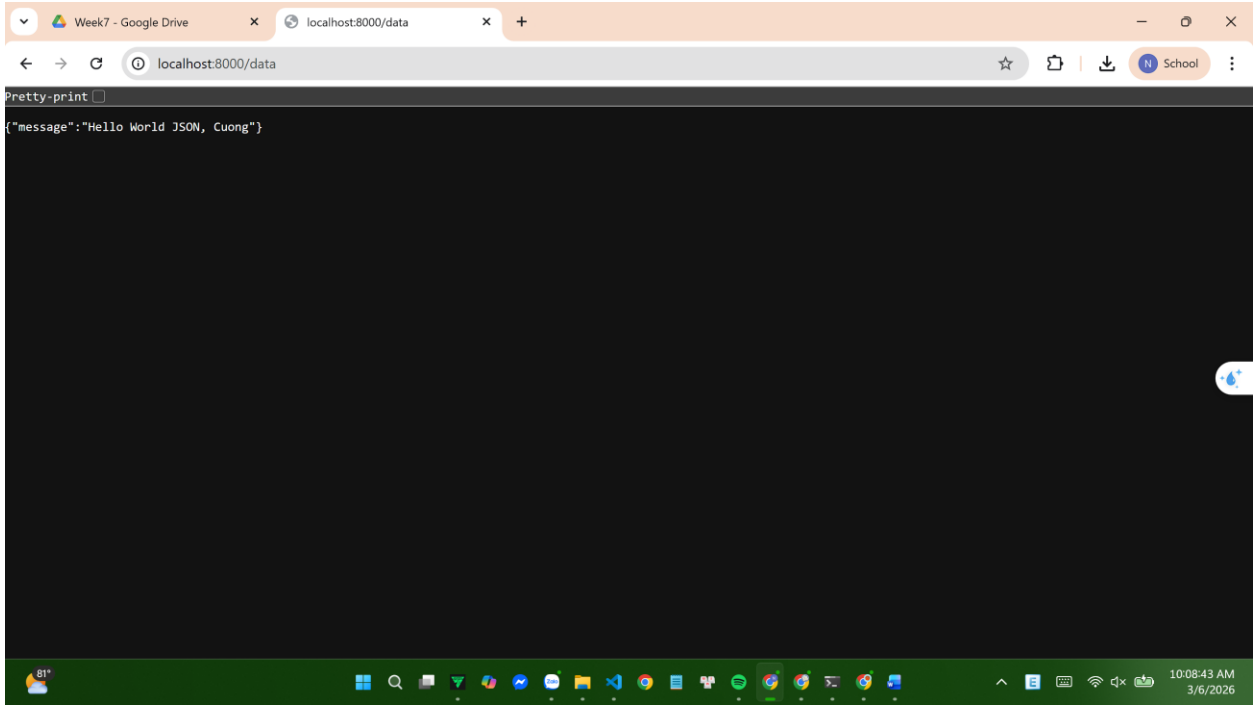
3. Screenshot 3:



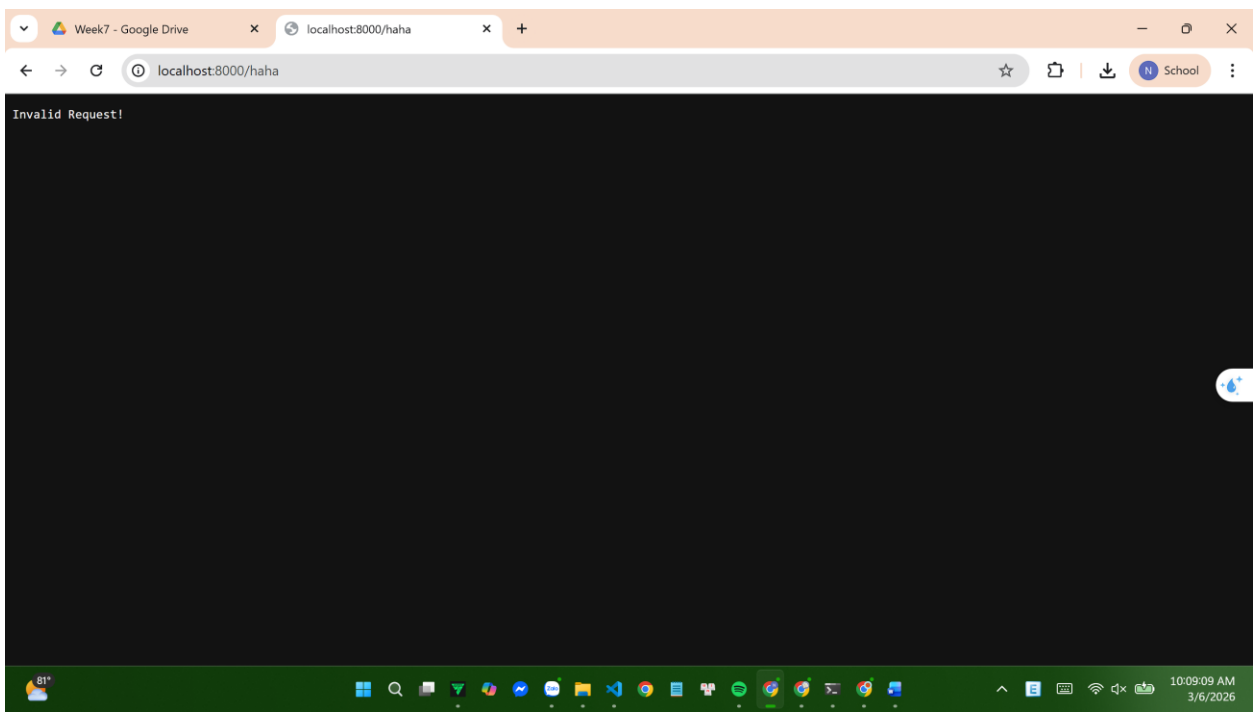
4. Screenshot 4:



5. Screenshot 5:



6. Screenshot 6:



7. Understanding part:

After reviewing and studying the documentation on Node.js servers, I gained a better understanding of how to use Node.js to create and run a simple web server. Node.js is a runtime environment that allows JavaScript to run outside the browser and is used to build server-side applications. A key point I learned is that Node.js is primarily used for network applications such as web servers and REST APIs. It works in a way that the server can handle multiple requests simultaneously without waiting for one task to complete before starting another. In this practice exercise, I learned how to create a simple Node.js server using the built-in HTTP module. The server is created by importing the HTTP module and using the createServer() function. This function listens for requests from clients and sends responses back to them. For example, when the server is running on port 8000, users can access it via localhost:8000 in their browser. Another important concept I learned was how Node.js handles HTTP requests using request (req) and response (res) objects. The request object provides information about the client's request, such as the URL, while the response object is used to send data back to the client. By examining req.url, the server can return different responses for different pages such as /, /student, /admin, or /data. The server can also return JSON data by setting the response header and sending the JSON content. Overall, this practice exercise helped me understand the basic structure of a Node.js web server and how it handles requests and responses.